

Name: Charlie Pyle
NUID: 22821006

Instructions Follow instructions *carefully*; failure to do so will result in points being deducted.

1. 16 points Using the **formal definition of Big-O**, show that each of these functions is $O(x^2)$. You must clearly state what the witnesses to the relationship are to receive full credit.

- (a) $f(x) = 26x$
- (b) $f(x) = 20x + 26$
- (c) $f(x) = 20x^2 + 2x + 6$
- (d) $f(x) = 20x^2 + 26\log(x)$

Solution: Enter your solution here:

A:

For $x \geq 1$ we have $x \leq x^2$, so

$$26x \leq 26x^2.$$

Thus $f(x) \in O(x^2)$ with witnesses $c = 26$, $k = 1$.

B:

For $x \geq 1$ we have $x \leq x^2$ and $1 \leq x^2$, so

$$20x + 26 \leq 20x^2 + 26x^2 = 46x^2.$$

Thus $f(x) \in O(x^2)$ with witnesses $c = 46$, $k = 1$.

C:

For $x \geq 1$ we have $x \leq x^2$ and $1 \leq x^2$, so

$$20x^2 + 2x + 6 \leq 20x^2 + 2x^2 + 6x^2 = 28x^2.$$

Thus $f(x) \in O(x^2)$ with witnesses $c = 28$, $k = 1$.

D:

For $x \geq 1$ we have $\log(x) \leq x \leq x^2$, so

$$20x^2 + 26\log(x) \leq 20x^2 + 26x^2 = 46x^2.$$

Thus $f(x) \in O(x^2)$ with witnesses $c = 46$, $k = 1$.

2. 8 points Using the **formal definition of Big-O**, show that $20x\log x$ is $O(26x^2)$ but that $26x^2$ is not $O(20x\log(x))$. You must clearly state what the witnesses to the relationship are to receive full credit.

Solution: Enter your solution here:

Part 1: $20x \log x \in O(26x^2)$ For $x \geq 1$ we have $\log(x) \leq x$, therefore

$$20x \log x \leq 20x \cdot x = 20x^2 \leq 26x^2.$$

Thus $20x \log x \in O(26x^2)$ with witnesses $c = 1$, $k = 1$.

Part 2: $26x^2 \notin O(20x \log x)$ Suppose for contradiction that $26x^2 \in O(20x \log x)$. Then there exist constants $c > 0$ and k such that $26x^2 \leq c \cdot 20x \log x$ for all $x \geq k$, which simplifies to

$$\frac{26x}{20c \log x} \leq 1 \quad \text{for all } x \geq k.$$

However,

$$\lim_{x \rightarrow \infty} \frac{26x}{20c \log x} = \frac{1}{c} \lim_{x \rightarrow \infty} \frac{26x}{20 \log x} = \infty,$$

a contradiction. Thus $26x^2 \notin O(20x \log x)$.

3. 16 points For each of the following functions, give a proof using the **Limit method** that $f(x) \in O(g(x))$:

- (a) $f(x) = 20x^2, g(x) = 26x^3$
- (b) $f(x) = 20x \log(x), g(x) = 26x^2$
- (c) $f(x) = 20 + \log(x) + 2x^2 + 6x^3, g(x) = x^3$
- (d) $f(x) = 20x \log(x) + 2x, g(x) = 6x^2$

Solution:

Enter your solution here:

A:

$$\lim_{x \rightarrow \infty} \frac{20x^2}{26x^3} = \frac{20}{26} \lim_{x \rightarrow \infty} \frac{1}{x} = 0.$$

Since the limit is $0 < \infty$, we conclude $f(x) \in O(g(x))$.

B: Divide numerator and denominator by x (valid for $x > 0$):

$$\lim_{x \rightarrow \infty} \frac{20x \log x}{26x^2} = \lim_{x \rightarrow \infty} \frac{20 \log x}{26x}.$$

Apply L'Hôpital's Rule:

$$\lim_{x \rightarrow \infty} \frac{20 \log x}{26x} = \lim_{x \rightarrow \infty} \frac{\frac{20}{x}}{26} = \lim_{x \rightarrow \infty} \frac{20}{26x} = 0.$$

Since the limit is $0 < \infty$, we conclude $f(x) \in O(g(x))$.

C:

$$\lim_{x \rightarrow \infty} \frac{20 + \log x + 2x^2 + 6x^3}{x^3} = \lim_{x \rightarrow \infty} \frac{20}{x^3} + \lim_{x \rightarrow \infty} \frac{\log x}{x^3} + \lim_{x \rightarrow \infty} \frac{2}{x} + \lim_{x \rightarrow \infty} 6 = 0 + 0 + 0 + 6 = 6.$$

Since the limit is $6 < \infty$, we conclude $f(x) \in O(g(x))$.

D:

$$\lim_{x \rightarrow \infty} \frac{20x \log x + 2x}{6x^2} = \lim_{x \rightarrow \infty} \frac{20 \log x}{6x} + \lim_{x \rightarrow \infty} \frac{2}{6x}.$$

The second term goes to 0. For the first term, apply L'Hôpital's Rule:

$$\lim_{x \rightarrow \infty} \frac{20 \log x}{6x} = \lim_{x \rightarrow \infty} \frac{\frac{20}{x}}{6} = \lim_{x \rightarrow \infty} \frac{20}{6x} = 0.$$

Therefore,

$$\lim_{x \rightarrow \infty} \frac{20x \log x + 2x}{6x^2} = 0 + 0 = 0.$$

Since the limit is $0 < \infty$, we conclude $f(x) \in O(g(x))$.

4. 8 points Answer the following parts:

- (a) Give pseudocode for an algorithm that finds the first repeated negative integer in a given sequence of integers.
- (b) Analyze the worst-case time complexity of the algorithm you devised in part (a). To receive full credit, you must state your choice of elementary operation.

Solution:

A:

```
1 function FirstRepeatedNegative(numbers):
2     seen = empty set
3     for x in numbers:
4         if x < 0:
5             if x in seen:
6                 return x
7             add x to seen
8     end
9     return "none"
```

B:

Elementary operation: Comparison (lines 3, 4, and 5). With n elements this gives at most $3n$ comparisons. Worst-case time complexity: $O(n)$.

5. 8 points Answer the following parts:

- (a) Give pseudocode for an algorithm that determines whether a string of n characters has all unique characters.
- (b) Analyze the worst-case time complexity of the algorithm you devised in part (a). To receive full credit, you must state your choice of elementary operation.

Solution:

A:

```
1 function UniqueString(s):
2     seen = empty set
3     for c in s:
4         if c in seen:
5             return false
6         add c to seen
7     end
```

```
8     return true
```

B:

Elementary operation: Comparison (lines 3 and 4). With n elements this gives at most $2n$ comparisons. Worst-case time complexity: $O(n)$.

6. 8 points Answer the following parts:

- (a) Give pseudocode for an algorithm that locates the last occurrence of the smallest element in a list of n integers. (Elements are not necessarily distinct.)
- (b) Analyze the worst-case time complexity of the algorithm you devised in part (a). To receive full credit, you must state your choice of elementary operation.

Solution:

A:

```
1 function LastMinIndex(ints):
2     min_val = ints[0]
3     index = 0
4     for i from 1 to n-1:
5         | if ints[i] <= min_val:
6         |     min_val = ints[i]
7         |     index = i
8     end
9     return index
```

B:

Elementary operation: Comparison (lines 4 and 5). The loop always runs from $i = 1$ to $i = n - 1$ regardless of the input. Each iteration incurs 2 comparisons: the loop condition on line 4 and the $\leq$ check on line 5, giving $2(n - 1)$ comparisons total. Worst-case time complexity: $O(n)$.

7. 8 points Answer the following parts:

- (a) Given an array of integers $nums$ of length n and an integer $target$, find three distinct indices i, j, k such that $nums[i] + nums[j] + nums[k] = target$.
You may assume exactly one such triplet exists, and you may not use the same element more than once. Return the indices in any order.
- (b) Analyze the worst-case time complexity of the algorithm you devised in part (a). To receive full credit, you must state your choice of elementary operation.

Solution:

A:

```
1 function ThreeSum(nums, target):
2     for i from 0 to n-1:
```

```

3      |      for j from i+1 to n-1:
4      |      |      for k from j+1 to n-1:
5      |      |      |      if nums[i] + nums[j] + nums[k] == target:
6      |      |      |      return (i, j, k)
7      |      |      end
8      |      end
9      end

```

B:

Elementary operation: Comparison (lines 2, 3, 4, and 5). The total number of comparisons is given by:

$$\sum_{i=0}^{n-1} \sum_{j=i+1}^{n-1} \sum_{k=j+1}^{n-1} 4$$

Evaluating the innermost sum:

$$\sum_{k=j+1}^{n-1} 4 = 4(n-1-j)$$

Substituting into the middle sum:

$$\sum_{j=i+1}^{n-1} 4(n-1-j) = 4 \sum_{m=0}^{n-2-i} m = 4 \cdot \frac{(n-1-i)(n-2-i)}{2} = 2(n-1-i)(n-2-i)$$

Substituting into the outer sum:

$$\sum_{i=0}^{n-1} 2(n-1-i)(n-2-i) = 2 \cdot \frac{n(n-1)(n-2)}{3} = \frac{2n(n-1)(n-2)}{3} = 4 \cdot \binom{n}{3} \in \Theta(n^3)$$

In the worst case every triple is examined, giving $\frac{2n(n-1)(n-2)}{3}$ total comparisons. Worst-case time complexity: $O(n^3)$.